

BÁO CÁO PHÂN TÍCH KIẾN TRÚC PHẦN MỀM

Codebase: Base Backend API

Xác định kiến trúc hệ thống, mẫu kiến trúc, mẫu thiết kế và mô hình hóa C4 Level 1 - Level 2

Môn học / chuyên đề	Kiến trúc phần mềm / Phân tích codebase backend
Đối tượng khảo sát	Dự án Node.js + Express + TypeScript trong repo TCNS-DCD-BACK
Phạm vi báo cáo	Phân tích trạng thái hiện tại của codebase, không giả định kiến trúc tương lai
Người biên soạn	Codex
Ngày lập	07/04/2026

Tài liệu này được biên soạn dựa trên mã nguồn, cấu trúc thư mục, dependency flow và hành vi triển khai hiện có của hệ thống.

MỤC LỤC

- 1. 1. Mở đầu
- 2. 2. Tổng quan hệ thống và phương pháp khảo sát
- 3. 3. Xác định kiến trúc hệ thống hiện tại
- 4. 4. Phân tích mẫu kiến trúc đang áp dụng
- 5. 5. Phân tích các mẫu thiết kế xuất hiện trong codebase
- 6. 6. Mô hình hóa kiến trúc phần mềm theo C4 Model
- 7. 7. Đánh giá ưu điểm, hạn chế và định hướng cải tiến
- 8. 8. Kết luận
- 9. Phụ lục A. Bản đồ module và file mốc
- 10. Phụ lục B. Ma trận pattern - vị trí áp dụng

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải nghĩa
API	Application Programming Interface
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
DI	Dependency Injection
CQRS	Command Query Responsibility Segregation
C4	Context, Container, Component, Code
OTP	One-Time Password

1. MỞ ĐẦU

Trong các dự án backend hiện đại, việc xác định đúng kiến trúc đang tồn tại quan trọng không kém việc thiết kế một kiến trúc mới. Nếu đánh giá sai bản chất hệ thống, nhóm phát triển dễ lựa chọn nhầm hướng refactor, áp dụng sai pattern, hoặc đặt ra yêu cầu triển khai vượt quá nhu cầu thực tế của sản phẩm. Báo cáo này được lập nhằm xác định codebase hiện tại của Base Backend API đang vận hành theo mô hình nào, từ đó giải thích rõ vì sao hệ thống phù hợp với kết luận đó hơn các lựa chọn như SOA, Microservice, CQRS hay Clean Architecture đầy đủ.

Đối tượng khảo sát là một mã nguồn backend sử dụng Node.js, Express và TypeScript. Hệ thống được tổ chức theo các module nghiệp vụ như auth, users, duty, files, notifications, reward-penalties và reports. Trong mỗi module, project tiếp tục phân tách thành routes, controllers, services, repositories, schemas và một số shared components dùng chung. Việc khảo sát không chỉ dừng ở cấu trúc thư mục mà còn xét đến endpoint khởi động, cách mount route, luồng xử lý request, cách service phối hợp repository và phương thức truy cập dữ liệu ở tầng database adapter.

Mục tiêu của báo cáo gồm bốn phần. Thứ nhất, xác định kiến trúc tổng thể ở cấp hệ thống: monolith, SOA hay microservice. Thứ hai, xác định mẫu kiến trúc đang được áp dụng ở cấp tổ chức nội bộ của mã nguồn: N-layered, MVC, CQRS hay Clean Architecture. Thứ ba, nhận diện các design pattern xuất hiện trong codebase và phân loại chúng theo ba nhóm chính là creational, structural và behavioral. Thứ tư, mô hình hóa kiến trúc hiện tại theo C4 Model ở Level 1 và Level 2, để thể hiện các thành phần chính, quan hệ giữa chúng và ranh giới với các hệ thống bên ngoài.

Phương pháp thực hiện là khảo sát tĩnh dựa trên mã nguồn thật. Các file mốc được dùng để đưa ra kết luận gồm endpoint khởi động, bootstrap Express app, router tổng, middleware dùng chung, base service, base repository, database adapter, cùng một số module đại diện như users, auth và files. Việc đánh giá ưu tiên dependency flow thực tế hơn tên gọi thư mục. Nói cách khác, hệ thống được xác định dựa trên cách nó thực sự chạy và phối hợp thành phần, chứ không chỉ dựa trên tài liệu mô tả hoặc ý định ban đầu của người thiết kế.

Báo cáo cố ý giữ góc nhìn thực dụng. Nếu một pattern chỉ xuất hiện ở mức một kỹ thuật hỗ trợ nhỏ, tài liệu sẽ nêu rõ phạm vi ảnh hưởng của pattern đó thay vì phóng đại thành kiến trúc chính. Tương tự, nếu project có dấu hiệu của dependency injection ở cấp constructor mặc định nhưng chưa có container, composition root hay vòng phụ thuộc kiểu Clean Architecture, báo cáo sẽ gọi đó là 'DI mức nhẹ' thay vì kết luận hệ thống đã là Clean Architecture. Cách tiếp cận này giúp tài liệu phản ánh trung thực tình trạng codebase hiện tại, phục vụ tốt hơn cho việc học tập, bảo trì và định hướng cải tiến.

2. TỔNG QUAN HỆ THỐNG VÀ PHƯƠNG PHÁP KHẢO SÁT

2.1. Công nghệ và thành phần nền tảng

Codebase hiện tại sử dụng Node.js kết hợp Express làm HTTP framework, TypeScript làm ngôn ngữ chính, Mongoose làm lớp tương tác với MongoDB, cùng một số thư viện hỗ trợ như jsonwebtoken cho xác thực, bcryptjs cho băm mật khẩu, multer và sharp cho xử lý upload, axios cho giao tiếp HTTP ra ngoài và swagger-ui-express cho tài liệu API. Tập hợp dependency này cho thấy dự án được định hướng như một backend API truyền thống, không phải dạng event-driven system hay distributed platform phức tạp.

Điểm khởi động của ứng dụng nằm ở file index.ts, nơi dotenv được nạp và server chính được import vào process. Tại server.ts, ứng dụng Express được tạo, cấu hình middleware bảo mật, CORS, parse body, static uploads, parse query, logging, rate limiting và error handling. Sau đó toàn bộ route được mount dưới tiền tố /api. Trình tự khởi động này cho thấy toàn bộ API chạy trong một tiến trình ứng dụng duy nhất, với một pipeline xử lý request thống nhất và một cơ chế khởi tạo database tập trung trước khi listen cổng mạng.

Router tổng trong src/routes/index.ts tiếp nhận các route module con như auth, users, upload, files, notifications, duty, reward-penalties và reports. Mỗi module giữ phần route riêng của mình rồi được ghép lại trong cùng một ứng dụng Express. Đây là dấu hiệu rất điển hình của modular monolith: chia code theo nghiệp vụ để dễ bảo trì, nhưng vẫn triển khai và vận hành như một hệ thống thống nhất, không có ranh giới deploy độc lập giữa các module.

Ở cấp module, project áp dụng cách tổ chức tương đối đồng đều: routes tiếp nhận request và gắn middleware; controllers nhận input và trả response; services chịu trách nhiệm chính về logic nghiệp vụ; repositories truy cập dữ liệu; schemas mô tả shape dữ liệu và ràng buộc. Ngoài ra, shared/common và shared/repositories chứa các base class và logic dùng lại toàn hệ thống. Cách chia này tạo nên sự cân bằng giữa tái sử dụng và cô lập nghiệp vụ, đồng thời phản ánh rõ triết lý thiết kế 'module theo domain nhỏ, layer theo trách nhiệm'.

2.2. Bản đồ thư mục mức cao

Vị trí	Vai trò chính	Nhận xét kiến trúc
index.ts	Entrypoint process	Chỉ có một endpoint cho toàn bộ backend
src/server.ts	Bootstrap Express app	Thể hiện pipeline middleware chung và một app duy nhất
src/routes	Router tổng và mount module	Cho thấy tổ chức theo module nghiệp vụ
src/modules/*	Auth, users, duty, files, notifications, reports...	Mỗi module là một khối nghiệp vụ trong monolith
src/shared/common	BaseController, BaseService, EntitySchemaService	Tái sử dụng logic tầng ứng dụng
src/shared/repositories	BaseRepository	Chuẩn hóa truy cập dữ liệu
src/database	Mongo adapter và init database	Tầng persistence/infrastructure

2.3. Luồng xử lý request điển hình

```
[Client / Frontend]
  |
  v
Route -> Middleware auth/validation/query
      -> Controller
      -> Service
      -> Repository
      -> Database Adapter
      -> MongoDB
      ^
      |
Response + Pagination headers + Error handling
```

Luồng trên được chứng minh rõ khi xem module users: route gắn `requireAuth`, `requirePermission`, `validateSchema`; controller chỉ điều phối input và response; service xử lý thống kê, chuẩn hóa dữ liệu, quan hệ với notification, duty và reward-penalty; repository thực hiện truy vấn chuyên biệt như `findByEmail` hoặc `updateLastLogin`;

cuối cùng BaseRepository ủy quyền cho DatabaseAdapter. Việc quan sát dependency flow như vậy là cơ sở mạnh nhất để nhận diện đúng kiến trúc và pattern của hệ thống.

2.4. Phương pháp kết luận kiến trúc

- Ưu tiên dependency flow thật thay vì tên gọi thư mục.
- Đối chiếu tài liệu README/docs với implementation đang chạy thực tế.
- Xác định pattern ở đúng cấp độ: system, module, class hay helper.
- Phân biệt rõ 'có dấu hiệu' với 'được áp dụng đầy đủ'.
- Ghi nhận cả những điểm lệch giữa tài liệu và code hiện hành.

3. XÁC ĐỊNH KIẾN TRÚC HỆ THỐNG HIỆN TẠI

3.1. Đặt vấn đề

Ba lựa chọn thường được đặt ra trong các bài phân tích backend là Monolith, SOA và Microservice. Tuy nhiên, các thuật ngữ này thường bị dùng lẫn lộn nếu chỉ nhìn vào việc project có nhiều module hay nhiều route. Thực chất, tiêu chí phân biệt nằm ở số tiến trình chạy độc lập, mức độ deploy tách rời, cơ chế giao tiếp nội bộ, quyền sở hữu dữ liệu và khả năng phát hành từng phần của hệ thống. Do đó, để xác định đúng, cần nhìn vào cách ứng dụng khởi động, cách module được ghép vào runtime và cách chúng truy cập dữ liệu.

3.2. So sánh các khả năng

Tiêu chí	Monolith	SOA	Microservice	Hiện trạng của Base Backend
Số ứng dụng deploy	Một	Nhiều service lớn	Nhiều service nhỏ	Một app Express duy nhất
Entrypoint runtime	Một tiến trình	Nhiều tiến trình	Nhiều tiến trình	index.ts -> server.ts -> app.listen
Giao tiếp nội bộ	Gọi hàm trực tiếp	HTTP/queue/ service bus	HTTP/gRPC/ queue	Import module trực tiếp trong cùng codebase
Quyền sở hữu dữ liệu	Thường dùng chung DB	Có thể tách	Ưu tiên DB riêng từng service	Repository cùng truy cập một database adapter
Triển khai độc lập	Không	Có	Có mạnh	Không có dấu hiệu deploy từng module riêng
Kết luận	Phù hợp	Không phù hợp	Không phù hợp	Modular Monolith

3.3. Vì sao đây là Monolith

Thứ nhất, ứng dụng chỉ có một entrypoint cho toàn hệ thống. File `index.ts` nạp cấu hình môi trường rồi import server chính; `server.ts` khởi tạo Express app, đăng ký middleware, mount routes và listen một cổng duy nhất. Nếu đây là microservice, thông thường sẽ có nhiều entrypoint, nhiều service package hoặc ít nhất nhiều tiến trình đóng vai trò khác nhau. Trong project này, không có bằng chứng nào cho thấy auth, users, files hay reports chạy thành service độc lập.

Thứ hai, các module nghiệp vụ giao tiếp với nhau bằng cách import trực tiếp service hoặc repository. Ví dụ user service có thể gọi `notificationsRepository`, `rewardPenaltiesRepository`, `dutySlotsRepository` và `dutySwapRequestsRepository` ngay trong cùng process. Kiểu phối hợp này rất tự nhiên trong monolith, nhưng sẽ không tồn tại theo cách đó trong microservice, nơi boundary giữa service đòi hỏi giao tiếp mạng hoặc message broker. Đây là bằng chứng dependency trực tiếp, mạnh hơn mọi tuyên bố ở tài liệu.

Thứ ba, route của tất cả module được mount vào cùng một router tổng dưới `/api`. Cấu trúc này cho phép hệ thống có nhiều khối nghiệp vụ tách biệt về mã nguồn nhưng không tách biệt về runtime. Nói cách khác, project này không phải 'monolith thuần túy dạng thư mục phẳng', mà là monolith được mô-đun hóa. Thuật ngữ chính xác hơn là modular monolith, vì sự phân chia theo module được thực hiện nghiêm túc nhưng vẫn ở trong một khối triển khai thống nhất.

Thứ tư, tầng dữ liệu được chuẩn hóa thông qua một `DatabaseAdapter` dùng chung. Mọi repository đều đi qua cùng abstraction này và cùng được khởi tạo bằng `initDatabase` trước khi server listen. Mặc dù abstraction đủ để cho phép thay adapter trong tương lai, bản chất runtime hiện tại vẫn là một hệ thống duy nhất, không phải một tập service tự trị với data ownership độc lập. Đây là đặc trưng quen thuộc của monolith tối ưu hóa maintainability chứ chưa phải kiến trúc phân tán.

Thứ năm, dù có giao tiếp HTTP ra ngoài như Cloudinary hoặc OTP gateway, đó chỉ là outbound integration với hệ thống bên ngoài. Sự tồn tại của external API không biến một monolith thành SOA hay microservice. Điều cần xét là ranh giới nội bộ của hệ thống chính. Trong codebase này, ranh giới nội bộ vẫn là ranh giới module trong cùng

repo và cùng process. Bởi vậy, kết luận hợp lý và chính xác nhất là Base Backend đang được xây dựng theo dạng modular monolith.

3.4. Vì sao không phải SOA hay Microservice

- Không có service registry, API gateway nội bộ hay message bus cho giao tiếp giữa module nghiệp vụ.
- Không có dấu hiệu tách repo hoặc package runtime cho từng service nghiệp vụ độc lập.
- Không có cơ chế phát hành auth/users/files như các deployment unit riêng.
- Không có ownership dữ liệu theo từng service; thay vào đó là repository dùng chung database abstraction.
- Việc import trực tiếp giữa các module cho thấy coupling ở cấp monolith, không phải coupling mạng giữa service.

Tóm lại, ở cấp kiến trúc hệ thống, phương án mô tả chính xác nhất là: dự án đang theo kiến trúc Monolith, cụ thể hơn là Modular Monolith. Đây là kết luận phù hợp với cách hệ thống đang chạy, cách code được tổ chức và cách thành phần phụ thuộc lẫn nhau trong runtime.

4. PHÂN TÍCH MẪU KIẾN TRÚC ĐANG ÁP DỤNG

4.1. Mẫu kiến trúc nổi trội: N-layered theo module

Bên trong modular monolith, mẫu kiến trúc nổi trội nhất là N-layered, hay cụ thể hơn là layered architecture được tổ chức theo module nghiệp vụ. Mỗi module không chỉ có route riêng mà còn có controller, service, repository và schema riêng. Điều này cho phép tách biệt rõ trách nhiệm: route quản lý endpoint và middleware; controller quản lý request/response; service chứa business logic; repository thao tác dữ liệu; database adapter cách ly chi tiết persistence. Luồng phụ thuộc đi từ ngoài vào trong và không đi ngược từ repository lên controller.

N-layered thể hiện rất rõ ở module users. Route định nghĩa endpoint như getAll, create, update, delete, profile và import/export. Controller nhận req, chọn service thích hợp, sanitize kết quả và trả res. Service xử lý các rule nghiệp vụ như hash password, đồng bộ avatar, thống kê người dùng, xóa quan hệ chéo với duty và notifications. Repository chỉ giữ các truy vấn chuyên biệt như tìm theo email, tìm theo phone candidate hoặc update last login. Cấu trúc này bền vững, dễ hiểu và phù hợp với backend CRUD có thêm business rule vừa phải.

Ngoài các module nghiệp vụ, project còn có shared/common và shared/repositories. Sự hiện diện của BaseController, BaseService và BaseRepository không phá vỡ layered architecture mà ngược lại còn củng cố nó, vì chúng chuẩn hóa hành vi ở từng tầng. Điều quan trọng là các base class này vẫn bám chặt vào vai trò của layer tương ứng: controller chỉ hỗ trợ HTTP, service chỉ hỗ trợ nghiệp vụ chung, repository chỉ hỗ trợ data access. Đây là biểu hiện của việc tái sử dụng đúng chỗ trong một kiến trúc phân tầng.

4.2. Phân biệt với MVC

Một số người có thể nhìn thấy Controller và Model rồi vội kết luận hệ thống là MVC. Tuy nhiên, MVC truyền thống nhấn mạnh ba thành phần Model - View - Controller, trong đó View là thành phần hiển thị trực tiếp với người dùng. Backend API hiện tại không có tầng View nội bộ; phản hồi chủ yếu là JSON và Swagger chỉ đóng vai trò tài liệu. Vì vậy, gọi hệ thống là MVC sẽ không sai hoàn toàn ở mức khái quát rất rộng,

nhưng không phải mô tả chính xác nhất. Mô tả tốt hơn vẫn là layered backend API, bởi trách nhiệm chính nằm ở tách HTTP layer, business layer và data layer.

Mẫu kiến trúc	Mức độ phù hợp	Giải thích ngắn
N-layered	Có	Route, controller, service, repository, database adapter tách rõ
MVC	Chỉ phù hợp một phần	Có controller nhưng không có view theo nghĩa kiến trúc web truyền thống
CQRS	Không	Không tách riêng command model và query model
Clean Architecture	Chưa đầy đủ	Có chút abstraction nhưng chưa có dependency rule chặt ở domain/application/infrastructure

4.3. Vì sao không phải CQRS

CQRS yêu cầu tách rạch ròi phần command và query, thường đi kèm model đọc/ghi khác nhau, handler riêng, hoặc ít nhất là design rõ ràng cho hai loại use case. Trong codebase này, đọc và ghi vẫn đi chung qua service và repository. BaseService cung cấp đồng thời findAll, findById, create, update, delete, search, bulk operations và import/export. Đây là dấu hiệu của service CRUD hợp nhất chứ không phải command/query tách biệt.

Middleware parseApiQuery chỉ chuẩn hóa query string và phân trang cho nhiều endpoint, nhưng điều đó không đồng nghĩa với CQRS. Nó đơn thuần là tiện ích hỗ trợ query ở cùng một flow service/repository đang có. Không tồn tại command handlers, query handlers, mediator hay projection store riêng. Vì thế, xếp project này vào CQRS sẽ gây hiểu lầm về mức độ phân tách thực sự của hệ thống.

4.4. Vì sao chưa phải Clean Architecture đầy đủ

Clean Architecture yêu cầu vòng phụ thuộc hướng vào trong, tách rõ entities, use cases, interface adapters và frameworks/infrastructure. Nó cũng thường kéo theo dependency injection rõ ràng, composition root, interface cho nhiều boundary và hạn chế mạnh việc inner layer biết đến outer layer. Trong project hiện tại, service vẫn import concrete repository, repository import concrete database proxy, controller import concrete service singleton. Điều này cho thấy dependency direction vẫn mang tính thực dụng của layered architecture hơn là tuân thủ chặt dependency rule của Clean Architecture.

Dù vậy, codebase có một vài tín hiệu tích cực liên quan đến Clean Architecture. BaseRepository phụ thuộc vào interface DatabaseAdapter thay vì gắn cứng vào mongoose API; AuthService và UploadService có constructor nhận dependency với giá trị mặc định; database được khởi tạo tập trung trước khi app listen. Tuy nhiên, các yếu tố này mới chỉ là 'abstraction hỗ trợ maintainability', chưa đủ để gọi hệ thống là clean architecture hoàn chỉnh. Kết luận thận trọng hơn là: dự án có một số kỹ thuật gần với clean architecture, nhưng kiến trúc chính vẫn là modular layered.

4.5. Ý nghĩa thực tiễn của kết luận

Việc xác định đúng mẫu kiến trúc giúp nhóm phát triển ra quyết định chính xác hơn. Với codebase hiện tại, ưu tiên hợp lý là tiếp tục củng cố modular layered architecture: giữ controller mỏng, gom rule nghiệp vụ vào service, giữ repository chỉ lo dữ liệu, kiểm soát shared layer để tránh trở thành 'kho chứa mọi thứ'. Nếu vội ép hệ thống sang CQRS hay Clean Architecture toàn diện khi quy mô hiện tại chưa đòi hỏi, chi phí refactor có thể lớn hơn lợi ích thực tế. Nói cách khác, hiểu đúng kiến trúc hiện tại là điều kiện để cải tiến đúng mức, không over-engineer.

5. PHÂN TÍCH CÁC MẪU THIẾT KẾ XUẤT HIỆN TRONG CODEBASE

5.1. Nhóm tạo dựng (Creational Patterns)

Trong codebase này không xuất hiện nổi bật các pattern tạo dựng theo nghĩa kinh điển như Abstract Factory, Builder hay Prototype. Cách tạo object chủ yếu là khởi tạo trực tiếp class rồi export một instance dùng chung. Nhiều controller, service và repository kết thúc bằng export default new ..., tạo ra singleton-style instance ở phạm vi module. Đây là một lựa chọn thực dụng, giúp tránh lặp khởi tạo và thuận tiện cho import xuyên suốt project.

Ngoài singleton-style instance, project có một mức dependency injection nhẹ ở constructor. Ví dụ AuthService nhận userRepository với giá trị mặc định, UploadService nhận policyService, processingService, storageService và recordService thông qua constructor default. Cách làm này hữu ích cho testability và tách phụ thuộc ở mức vừa phải, nhưng chưa đạt mức DI container hay IoC framework. Vì vậy, ở nhóm tạo dựng, báo cáo ghi nhận hai điểm chính: singleton-style module instance và constructor-based injection ở mức đơn giản.

5.2. Nhóm cấu trúc (Structural Patterns)

5.2.1. Repository Pattern

Repository Pattern là pattern nổi bật nhất ở nhóm cấu trúc. BaseRepository chuẩn hóa thao tác findAllAdvanced, findById, findOne, findMany, create, update, delete, count, exists và distinct. Các repository cụ thể như UsersRepository kế thừa BaseRepository rồi bổ sung các truy vấn chuyên biệt. Nhờ vậy, service không làm việc trực tiếp với Mongoose Model mà tương tác thông qua một lớp repository có interface ổn định hơn. Cách làm này giúp giảm coupling giữa business logic và persistence technology, đồng thời tập trung query logic ở đúng nơi.

5.2.2. Adapter Pattern

Adapter Pattern xuất hiện ở tầng database. Thay vì để repository gọi thẳng mongoose API, project định nghĩa interface DatabaseAdapter rồi cung cấp MongoConnect như

một implementation cụ thể. BaseRepository chỉ cần biết database có các hàm findAllAdvanced, findById, create, update, delete, count và các utility khác. Đây là một biểu hiện rõ ràng của adapter: cùng một contract được dùng để che giấu chi tiết implementation của persistence. Dù runtime hiện tại đã cố định vào MongoDB, abstraction này vẫn có giá trị kiến trúc vì nó làm mềm phụ thuộc ở tầng data access.

5.2.3. Facade-like orchestration ở upload module

UploadService đóng vai trò gần với Facade hơn là một service CRUD thuần túy. Nó gom nhiều thành phần: uploadPolicyService để dựng middleware upload, fileProcessingService để chuẩn hóa file đầu vào, cloudinaryStorageService để làm việc với Cloudinary và fileService để lưu metadata record. Từ góc nhìn controller, upload avatar hay upload general file chỉ là một lời gọi gọn vào UploadService, trong khi bên trong service này điều phối nhiều bước. Đây là một ví dụ tốt của facade-like pattern ở cấp module: cung cấp một giao diện đơn giản cho một chuỗi xử lý phức hợp hơn bên trong.

5.2.4. Base class reuse trong controller

BaseController không phải một design pattern kinh điển bắt buộc phải gọi tên, nhưng nó là một cơ chế tái sử dụng cấu trúc ở HTTP layer. Các handler CRUD cơ bản như getAll, getById, create, update, delete, search, count, bulk và validate được gom vào base class; controller cụ thể chỉ override hoặc mở rộng khi có nhu cầu nghiệp vụ riêng. Cách dùng kế thừa này giúp giảm lặp code và giữ controller nhất quán. Tuy nhiên, nó cũng cần được kiểm soát để tránh ép mọi use case vào cùng một bộ generic handler.

5.3. Nhóm hành vi (Behavioral Patterns)

5.3.1. Template Method

BaseService là ví dụ điển hình cho Template Method. Các hàm create, update và delete trong base service định nghĩa sẵn một quy trình: kiểm tra tồn tại, validate theo schema, validate tùy biến, beforeCreate/beforeUpdate/beforeDelete, gọi repository, rồi afterCreate/afterUpdate/afterDelete. Service con như UserService chỉ cần override các hook cần thiết để tùy biến hành vi mà không phá vỡ flow chuẩn. Pattern này rất phù

hợp với các module có nhiều CRUD chung nhưng vẫn cần chèn rule riêng ở từng bước.

5.3.2. Chain of Responsibility

Chain of Responsibility xuất hiện rõ trong pipeline middleware của Express. Một request có thể đi qua helmet, CORS, parse body, camelizeBody, logRequest, wrapJson, appendPaginationHeaders, parseApiQuery, requireAuth, requirePermission, validateSchema rồi mới tới controller. Mỗi middleware nhận request, xử lý một phần trách nhiệm rồi chuyển tiếp bằng next(). Cấu trúc này giúp project bổ sung hoặc thay đổi quy tắc kiểm tra đầu vào mà không làm controller nặng lên, đồng thời là pattern hành vi rất tự nhiên trong ứng dụng web dùng middleware chain.

5.3.3. Service Layer

Service Layer không nằm trong nhóm GoF cổ điển nhưng là pattern nghiệp vụ rất rõ nét của codebase. Logic như hash password, tính thống kê người dùng, xóa dữ liệu liên quan, xử lý upload rollback, phát OTP, reset password hay kiểm tra quyền truy cập profile đều được đặt trong service thay vì controller hoặc repository. Việc tách này làm cho controller mỏng, repository gọn và business rule tập trung hơn. Đây là một quyết định đúng với dự án backend có nhiều rule nhưng chưa đủ lớn để cần use-case layer riêng kiểu clean architecture.

5.4. Ma trận pattern chính

Nhóm pattern	Pattern	Vị trí áp dụng	Ý nghĩa
Creational	Singleton-style module instance	Nhiều file export default new ...	Dùng chung một instance theo module
Creational	Constructor injection nhẹ	AuthService, UploadService	Cho phép thay dependency ở mức đơn giản
Structural	Repository	BaseRepository và repositories từng module	Tách business logic khỏi data access
Structural	Adapter	DatabaseAdapter +	Che giấu chi tiết

		MongoConnect	persistence
Structural	Facade-like	UploadService	Gói nhiều dịch vụ xử lý file vào một giao diện gọn
Behavioral	Template Method	BaseService	Khung CRUD chuẩn + hook override
Behavioral	Chain of Responsibility	Middleware Express	Xử lý request theo chuỗi trách nhiệm
Behavioral	Service Layer	Các service nghiệp vụ	Tập trung business logic

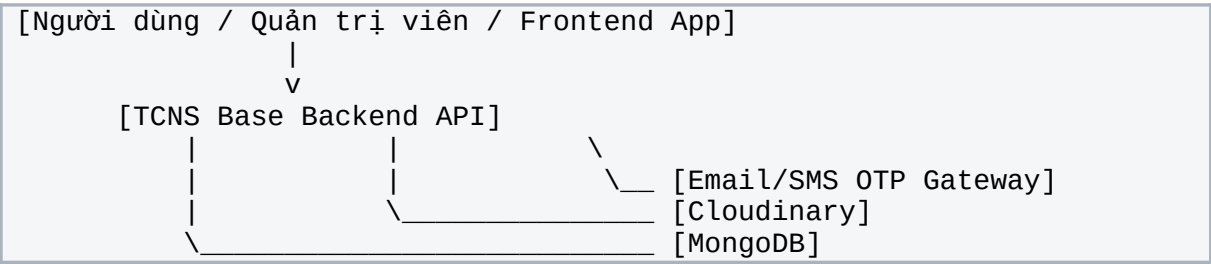
Nhìn tổng thể, codebase sử dụng design pattern theo hướng thực dụng và dễ bảo trì. Các pattern được áp dụng vì nhu cầu tổ chức code và kiểm soát phụ thuộc, chứ không phải để 'trang trí kiến trúc'. Đây là điểm tích cực vì nó giúp project giữ được độ đơn giản cần thiết trong khi vẫn có cấu trúc rõ ràng cho quá trình mở rộng sau này.

6. MÔ HÌNH HÓA KIẾN TRÚC PHẦN MỀM THEO C4 MODEL

6.1. Lý do sử dụng C4 Model

C4 Model là cách mô tả kiến trúc theo mức độ trừu tượng tăng dần, giúp người đọc nắm hệ thống từ tổng quan tới chi tiết. Với yêu cầu hiện tại, Level 1 và Level 2 là phù hợp nhất. Level 1 trả lời câu hỏi: hệ thống này nằm trong bối cảnh nào, ai sử dụng nó, và nó giao tiếp với hệ thống bên ngoài nào. Level 2 trả lời câu hỏi: bên trong hệ thống đang có những container nào và quan hệ giữa chúng ra sao. Vì project là modular monolith, Level 2 sẽ không phân rã thành nhiều ứng dụng nghiệp vụ độc lập mà tập trung vào Express API monolith và các thành phần infrastructure gắn với nó.

6.2. C4 Level 1 - System Context



Thành phần	Loại	Vai trò trong bối cảnh hệ thống
Người dùng / Admin / Frontend	Actor	Gửi request HTTP để xác thực, quản lý user, duty, upload, thông báo và báo cáo
TCNS Base Backend API	System	Xử lý toàn bộ nghiệp vụ backend trong một ứng dụng Node.js + Express
MongoDB	External system / database	Lưu trữ dữ liệu nghiệp vụ qua Mongo adapter
Cloudinary	External service	Lưu file, avatar, media và cung cấp thông tin/cleanup tài nguyên
Email/SMS OTP Gateway	External service	Gửi OTP cho quy trình quên

		mật khẩu
--	--	----------

Ở Level 1, hệ thống được nhìn như một khối backend thống nhất. Các external system chỉ xuất hiện ở vai trò dịch vụ bên ngoài được backend gọi tới. Điều này phù hợp với kết luận monolith: Cloudinary và OTP gateway là tích hợp outbound, không phải các service nội bộ cùng tham gia cấu thành backend theo kiểu microservice.

6.3. C4 Level 2 - Container Diagram

Container 1: Express API Monolith - routes - middleware - controllers - services - repositories - swagger docs ---> Container 2: MongoDB ---> External: Cloudinary ---> External: OTP Gateway
--

Container / hệ thống liên quan	Công nghệ	Trách nhiệm chính	Quan hệ
Express API Monolith	Node.js, Express, TypeScript	Xử lý request, nghiệp vụ, phân quyền, upload, báo cáo, import/export	Trung tâm của hệ thống
MongoDB	MongoDB + Mongoose	Lưu dữ liệu users, notifications, duty, reward-penalties, files...	Được truy cập qua DatabaseAdapter
Cloudinary	HTTP API	Lưu trữ file và metadata media từ module files/upload	Được gọi bởi cloudinaryStorageService
Email/SMS OTP Gateway	HTTP API	Phát OTP cho quy trình reset password	Được gọi bởi otpDeliveryService
Swagger UI	swagger-ui-express	Tài liệu hóa các endpoint trong cùng	Phục vụ như một thành phần hỗ trợ

		monolith	trong app
--	--	----------	-----------

Do hệ thống là monolith, Express API Monolith nên được xem là container trung tâm duy nhất ở Level 2. Các folder routes, controllers, services và repositories thực chất gần với Level 3 component hơn là các container tách rời. Nói cách khác, nếu tiếp tục mô hình hóa ở Level 3 thì khi đó mới hợp lý để vẽ cụ thể các thành phần như UsersController, UserService, UsersRepository, AuthService hay CloudinaryStorageService bên trong container Express API Monolith.

6.4. Mối liên hệ giữa C4 và kết luận kiến trúc

C4 Level 1 cho thấy người dùng chỉ tương tác với một backend duy nhất. Không có actor nào phải gọi riêng auth-service, user-service hay report-service. Đây là tín hiệu đồng thuận với kết luận modular monolith. Nếu hệ thống là microservice, Level 1 hoặc Level 2 thường sẽ lộ rõ nhiều service backend, API gateway hoặc message broker.

C4 Level 2 cho thấy container duy nhất của nội bộ hệ thống là Express API Monolith, còn MongoDB, Cloudinary và OTP gateway nằm ở phía ngoài. Điều này giúp người đọc không nhầm các external integration thành container nội bộ. Bài học rút ra là: trong phân tích kiến trúc, phải phân biệt ranh giới 'bên trong hệ thống' và 'hệ thống bên ngoài', nếu không rất dễ gán nhầm project monolith thành một kiến trúc phân tán.

7. ĐÁNH GIÁ ƯU ĐIỂM, HẠN CHẾ VÀ ĐỊNH HƯỚNG CẢI TIẾN

7.1. Ưu điểm của kiến trúc hiện tại

- Dễ hiểu và dễ onboarding cho thành viên mới nhờ dependency flow quen thuộc.
- Module hóa tốt theo nghiệp vụ, giảm cảm giác 'thư mục phẳng' của monolith truyền thống.
- Controller nhìn chung mỏng; business logic tập trung tương đối hợp lý ở service.
- Repository và database adapter giúp giảm coupling trực tiếp với persistence framework.
- Shared layer tái sử dụng đúng nhiều phần lặp như CRUD base, schema validation và import/export.
- Phù hợp với quy mô backend CRUD + business rules vừa phải, chưa cần chi phí vận hành microservice.

7.2. Hạn chế và rủi ro

Dù có module hóa, hệ thống vẫn là một khối deploy duy nhất. Điều đó nghĩa là khi một module tăng tải hoặc thay đổi lớn, toàn bộ ứng dụng vẫn cần build, test và phát hành cùng nhau. Đây không phải vấn đề nghiêm trọng ở quy mô hiện tại, nhưng cần được nhận thức rõ để tránh nhầm rằng project đã có các lợi ích vận hành của microservice.

BaseService hiện ôm khá nhiều hành vi generic: CRUD, batch operations, import/export và schema-based transformation. Trong giai đoạn đầu, điều này tiết kiệm thời gian. Tuy nhiên, nếu business rule mỗi module ngày càng khác nhau, việc tiếp tục nhồi use case vào base service có thể làm code khó đọc, hook chồng chéo và flow không còn rõ nghĩa. Khi đó, generic reuse quá rộng sẽ trở thành gánh nặng bảo trì.

Một điểm đáng chú ý là tài liệu môi trường vẫn nhắc `DB_CONNECTION=json | mongodb`, trong khi runtime hiện tại đã chỉ export Mongo adapter và `initDatabase` yêu cầu `DATABASE_URL`. Sự lệch này không làm hệ thống sai về chức năng, nhưng nó là rủi ro tài liệu: người mới đọc README hoặc `.env.example` có thể hiểu rằng project

còn hỗ trợ nhiều kiểu storage ở runtime, trong khi code thực tế không phản ánh điều đó đầy đủ.

7.3. Đề xuất cải tiến theo thứ tự ưu tiên

Mức ưu tiên	Đề xuất	Lợi ích chính	Ghi chú
Cao	Đồng bộ README/.env/docs với runtime Mongo hiện tại	Giảm hiểu nhầm cho người phát triển mới	Ít rủi ro, chi phí thấp
Cao	Giữ controller mỏng và hạn chế nhét business logic vào route/controller	Bảo toàn layered architecture	Nên xem là nguyên tắc phát triển
Trung bình	Chia nhỏ các service quá lớn theo nhóm use case trong cùng module	Dễ đọc hơn, bám nghiệp vụ hơn	Không nhất thiết tách thêm layer
Trung bình	Rà soát BaseService để chỉ generic hóa phần thực sự lặp	Giảm over-abstraction	Nên làm dần theo module
Thấp	Tăng mức DI/composition nếu cần test độc lập sâu hơn	Cải thiện testability	Chỉ nên làm khi có nhu cầu rõ
Thấp	Cân nhắc CQRS hoặc clean architecture cho module quá phức tạp	Tăng khả năng mở rộng ở điểm nóng	Không nên áp dụng đại trà toàn hệ thống

7.4. Định hướng kiến trúc phù hợp trong tương lai

Hướng đi hợp lý nhất cho giai đoạn tiếp theo không phải là nhảy ngay từ modular monolith sang microservice, mà là tiếp tục làm sạch ranh giới module và layer. Điều này bao gồm: giảm coupling xuyên module không cần thiết, chuẩn hóa input/output

cho service, tách service lớn thành các file nhỏ theo use case, giữ repository chỉ lo query và giảm số trường hợp 'shared' bị lạm dụng như nơi đặt business logic chéo.

Nếu trong tương lai xuất hiện nhu cầu mở rộng mạnh ở một số domain như upload/media processing hoặc reporting, khi đó có thể cân nhắc tách riêng các điểm nóng thành service độc lập. Tuy nhiên, việc tách nên dựa trên áp lực thật về scale, security boundary, release cadence hoặc ownership đội ngũ, chứ không nên dựa trên mong muốn 'kiến trúc cho hiện đại'. Hiện tại, modular monolith với layered architecture vẫn là lựa chọn phù hợp, tiết kiệm và dễ kiểm soát hơn cho codebase này.

8. KẾT LUẬN

Sau khi khảo sát entypoint, luồng bootstrap server, router tổng, cấu trúc module, base classes, repository, database adapter và các service tiêu biểu, có thể kết luận rằng Base Backend API hiện tại đang theo kiến trúc Monolith, chính xác hơn là Modular Monolith. Hệ thống được mô-đun hóa theo domain nghiệp vụ nhưng vẫn chạy trong một ứng dụng Node.js + Express duy nhất và dùng một quá trình khởi động tập trung.

Ở cấp tổ chức bên trong, mẫu kiến trúc chi phối là N-layered / Modular Layered Architecture với chuỗi trách nhiệm rõ ràng: Route -> Controller -> Service -> Repository -> Database Adapter -> MongoDB. Project không phải CQRS, không phải Clean Architecture đầy đủ và chỉ mang hơi hướng MVC ở mức có controller/model nhưng không có view nội bộ. Cách mô tả đúng nhất vì thế là modular monolith với layered architecture theo module nghiệp vụ.

Về design patterns, codebase thể hiện rõ Repository Pattern, Adapter Pattern, Template Method, Chain of Responsibility, Service Layer và một số kỹ thuật tái sử dụng như singleton-style instance và facade-like orchestration ở upload module. Các pattern này được áp dụng chủ yếu để kiểm soát phụ thuộc, tái sử dụng hợp lý và giữ code dễ hiểu, thay vì phục vụ một framework kiến trúc quá nặng.

Về mô hình hóa C4, ở Level 1 hệ thống xuất hiện như một backend API duy nhất tương tác với người dùng/frontend và ba external systems chính là MongoDB, Cloudinary, Email/SMS OTP gateway. Ở Level 2, container trung tâm vẫn là Express API Monolith; các phần như route, controller, service và repository nên được xem là component nội bộ hơn là container tách biệt. Kết luận này nhất quán với toàn bộ bằng chứng từ codebase.

Nhìn chung, kiến trúc hiện tại là hợp lý đối với quy mô và tính chất nghiệp vụ của dự án. Hướng cải tiến tối ưu không nằm ở việc thay tên kiến trúc cho 'hiện đại' hơn, mà nằm ở việc tiếp tục làm rõ module boundary, kiểm soát generic base class và đồng bộ tài liệu với implementation. Khi đó, hệ thống sẽ giữ được ưu điểm của modular monolith: đơn giản trong vận hành, rõ ràng trong phát triển và đủ linh hoạt để mở rộng khi nhu cầu thật sự xuất hiện.

PHỤ LỤC A. BẢN ĐỒ MODULE VÀ FILE MỐC

File / thư mục	Vai trò	Giá trị phân tích
index.ts	Entrypoint	Chứng minh chỉ có một tiến trình khởi động chính
src/server.ts	Bootstrap Express	Chứng minh pipeline middleware và router chung
src/routes/index.ts	Router tổng	Chứng minh các module được mount trong cùng app
src/modules/users/routes/ user.routes.ts	Module users - route layer	Thể hiện N-layered ở module điển hình
src/modules/users/ controllers/user.controller.ts	Controller layer	Cho thấy controller mỏng và tập trung HTTP
src/modules/users/services/ user.service.ts	Service layer	Cho thấy business logic và orchestration
src/modules/users/ repositories/ users.repository.ts	Repository layer	Truy vấn chuyên biệt nhưng vẫn bám BaseRepository
src/shared/common/base- service.ts	Base service	Template Method, generic CRUD, import/export
src/shared/repositories/ base.repository.ts	Base repository	Repository pattern và abstraction data access
src/types/database.ts	DatabaseAdapter contract	Mình chứng Adapter Pattern
src/database/mongo- database.adapter.ts	Mongo implementation	Mình chứng runtime hiện tại chỉ dùng MongoDB
src/modules/files/services/ upload.service.ts	Upload orchestration	Ví dụ facade-like service
src/modules/auth/services/ otp-delivery.service.ts	External integration	Mình chứng C4 boundary với OTP gateway

PHỤ LỤC B. MA TRẬN PATTERN - VỊ TRÍ ÁP DỤNG

Pattern	Nhóm	Ví dụ áp dụng	Nhận xét
Repository	Structural	BaseRepository, UsersRepository	Áp dụng nhất quán và có giá trị rõ
Adapter	Structural	DatabaseAdapter, MongoConnect	Tách data access khỏi chi tiết persistence
Template Method	Behavioral	BaseService create/update/delete	Phù hợp CRUD dùng chung
Chain of Responsibility	Behavioral	Middleware Express	Rất tự nhiên trong HTTP pipeline
Service Layer	Behavioral / enterprise	AuthService, UserService, UploadService	Tập trung nghiệp vụ đúng chỗ
Singleton-style instance	Creational	export default new ...	Thực dụng nhưng chưa phải singleton formal
Constructor injection nhẹ	Creational	AuthService, UploadService	Hữu ích cho testability, chưa phải DI container
Facade-like	Structural	UploadService	Che bớt độ phức tạp của nhiều dịch vụ xử lý file

PHỤ LỤC C. MÔ TẢ NGẮN GỌN CÓ THỂ DÙNG TRONG THUYẾT TRÌNH

- Kiến trúc hệ thống: Modular Monolith.
- Mẫu kiến trúc: Modular Layered / N-layered.
- Không phải Microservice, SOA, CQRS hay Clean Architecture đầy đủ.
- Pattern chính: Repository, Adapter, Template Method, Chain of Responsibility, Service Layer.

- C4 Level 1: Người dùng/Frontend -> Base Backend API -> MongoDB/Cloudinary/OTP Gateway.
- C4 Level 2: Express API Monolith là container trung tâm; MongoDB là database; Cloudinary và OTP gateway là external systems.